

POINTER

- ① It is a special kind of variable which can store the Address of any variable as well as any array. It will allocate ^{access} the memory ~~Address~~ (RAM) directly, ~~Pointer ty~~

Pointer type variable can hold the address of any variable as well as store the value of that variable. To store the address ' & ' is used and to access of the value of that variable through pointer ' * ' is used.

Integer type of pointer can only hold the Integer Address of integer type of variable and array, like wise float type of pointer and character type of pointer can only hold the Address of float type and character type of variable and array respectively.

② □ Array of pointer

When a pointer points to an array it is known as array of pointer, that means the pointer will hold the base address of ^{the} array

③ □ Pointer of ^{an} Array

When we ~~the~~ declare an array of pointer type, then that array it-self work as an array as well as it can work as a pointer to hold the base address of it self, it is known as pointer of an array.

④ □ Array

Array is not a dynamic memory allocated data type. It is known as static memory allocated data type, because it can allocate ~~memory~~ memory at compile time not runtime.

⑤ □ Pointer of pointer

When a pointer points to the address of another pointer it is known as pointer of pointer. Basically pointer of pointer holds the Address of ~~B~~ base variable via the 1st / base pointer

⑥ □ Dynamic memory allocation (DMA)

As we know array is not allocate memory at runtime so to allocate memory at runtime, Dynamic memory allocation concept or pointer concept has ~~been~~ introduced.

dynamic memory allocation consist of 4 inbuilt func on system default func.

① ~~malloc~~ malloc()

② calloc()

③ realloc()

④ free()

⑦ □ malloc()

It is a system defined or inbuilt func which is present in $\langle \text{malloc.h} \rangle$. It is used to allocate memory at runtime at a heap that means it will allocate memory at a contiguous memory location and each block is connect to each other.

It will take one parameter as an argument ~~that~~ i.e. no. of bytes are required to allocate memory at runtime.

⑧ □ calloc()

It is a system defined or inbuilt func which is present $\langle \text{malloc.h} \rangle$. It is used to allocate memory dynamically but not at a heap that means each block is not connect to each other.

It will take two parameter as an argument. 1st parameter how many block and 2nd parameter bytes required for each data type.

⑨ □ realloc()

It is a system defined or inbuilt func which is present in $\langle \text{malloc.h} \rangle$. It is used to resize the memory which is ~~allocated~~ ~~per~~ previously allocated by malloc or calloc func.

It will take one parameter as ~~an~~ argument ~~that~~ i.e. no. of bytes are required to allocate memory.

Disadvantage of dynamic memory allocation:

- ① if we allocate memory by using malloc, calloc or realloc func the memory will be allocated permanently within the cache memory, so if we power off the computer the cache memory will not be clear.

To solve the above problem ~~free()~~ free() is used.

⑩ free()

It is a system defined or inbuilt func which is present within <malloc.h>. It is used to deallocate the memory or clear the memory which is previously allocated by malloc, calloc or realloc func.

It will take one parameter as an argument. That is the pointer as an argument which is allocated at runtime.

DMA (Dynamic memory allocation)

- ① argc (argument count)
- ② argv (argument value)

① argc: It is a system defined variable which is passed as 1st argument within the main func.

It is used to count how many value present within the argv array. It is an integer type of variable.

② argv: It is a pointer type of array of character type.

It is also a system defined or inbuilt variable which is passed as an argument. It will take string values within each index.

Pointer

```
int X, *P;
```

```
printf("Enter the No. = ");
```

```
scanf("%d", &X);
```

$P = \underline{\underline{\&X}};$

P will hold the address of X variable & as well as hold the value of X.

RAM

X
15

100

100

*P 104

```
printf("value = %d", *P);
```

```
printf("Address = %u", &P);
```

Address of itself

```
printf("Address = %u", P);
```

Address of X via P pointer

Array of pointers

int a[5] = { 10, 20, 30, 40, 50 };

int *p;

p = &a[0];

a[0]	104	108	112	116
10	20	30	40	50
0	1	2	3	4
p				

printf("value = %d", *p);

↓
10

printf("Address = %u", p);

↓
100

printf("Address = %u", (p+2));

↓
108

pointer of pointer

```
int x, *p, **p1;
```

```
printf("Enter the no. = ");
```

```
scanf("%d", &x);
```

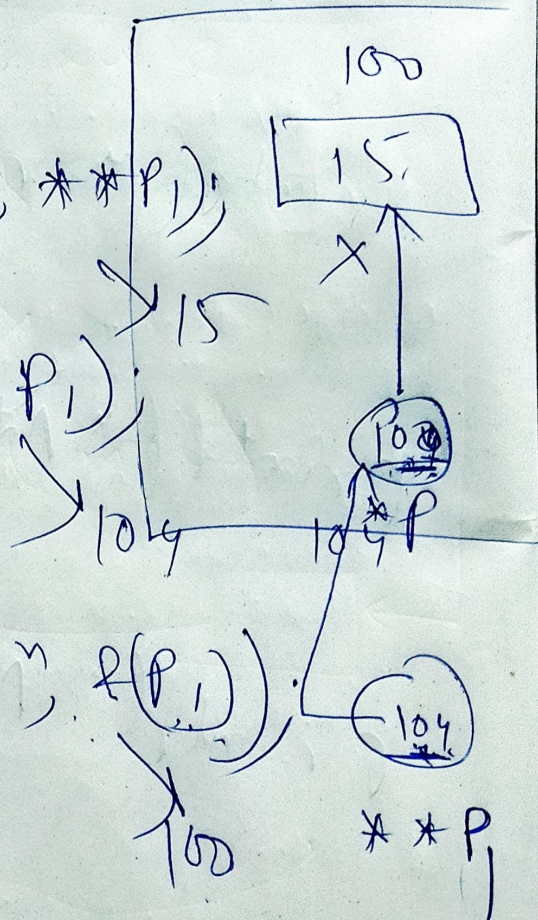
```
p = &x;
```

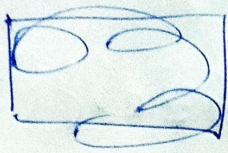
```
p1 = &p;
```

```
printf("value = %d", **p1);
```

```
printf("Address  
= %u", p1);
```

```
printf("Address  
= %u", &(p1));
```

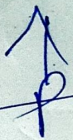
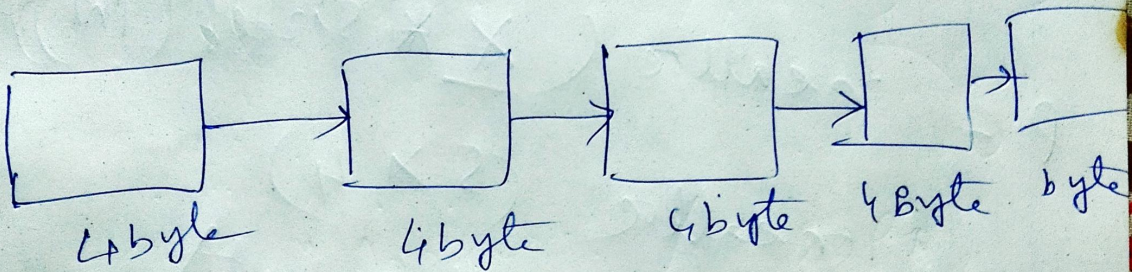




`int *p, x = 5;`

`p = malloc( X * sizeof(int) );`
 $\downarrow$
`= malloc( 5 * 4 byte )`

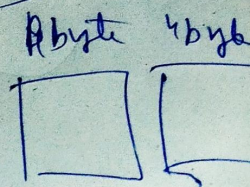
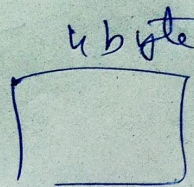
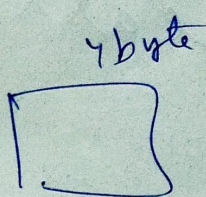
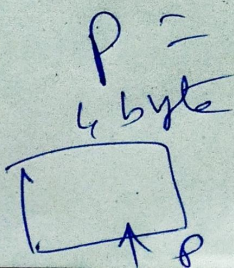
`p = 20 byte.`



`int *p, x = 5;`

`p = calloc( X, sizeof(int) );`

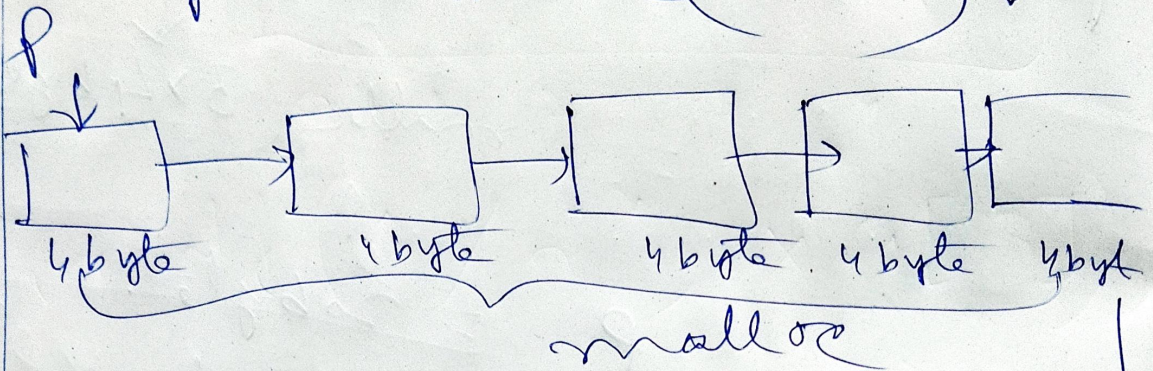
$\downarrow$
`calloc( 5, 4 byte )`



int *p, x=5;

p = malloc(X * sizeof(int));
(5 * 4 byte)

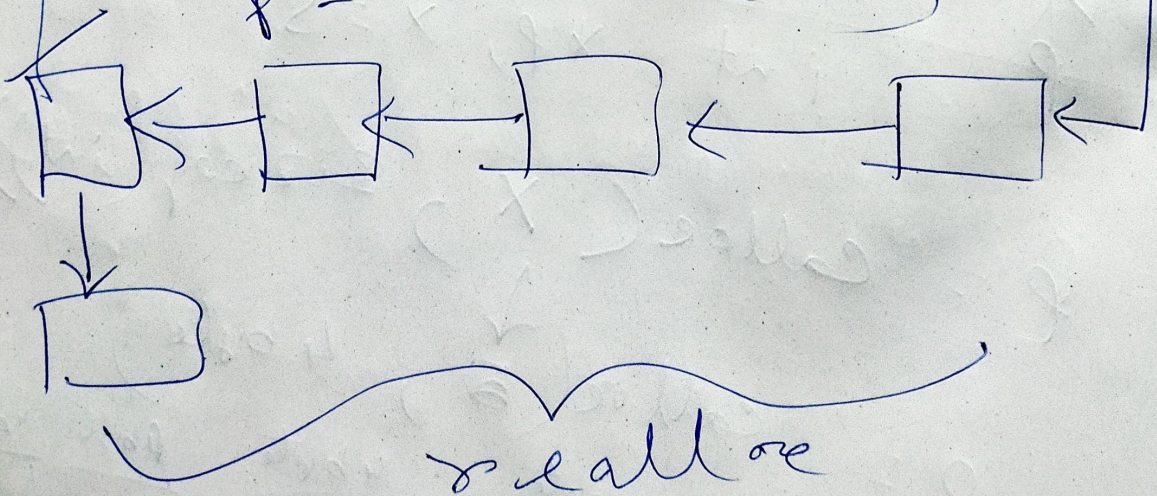
p = malloc(20 byte) ↓



p = realloc(X * sizeof(int));

realloc (5 * 4)

p = realloc(20 byte)



free(p);

```
void main(int argc,  
char *argv[])
```

```
{
```

```
}
```

anitra	sunayan	sampat
--------	---------	--------

*argv[0] argv[1] argv[2]

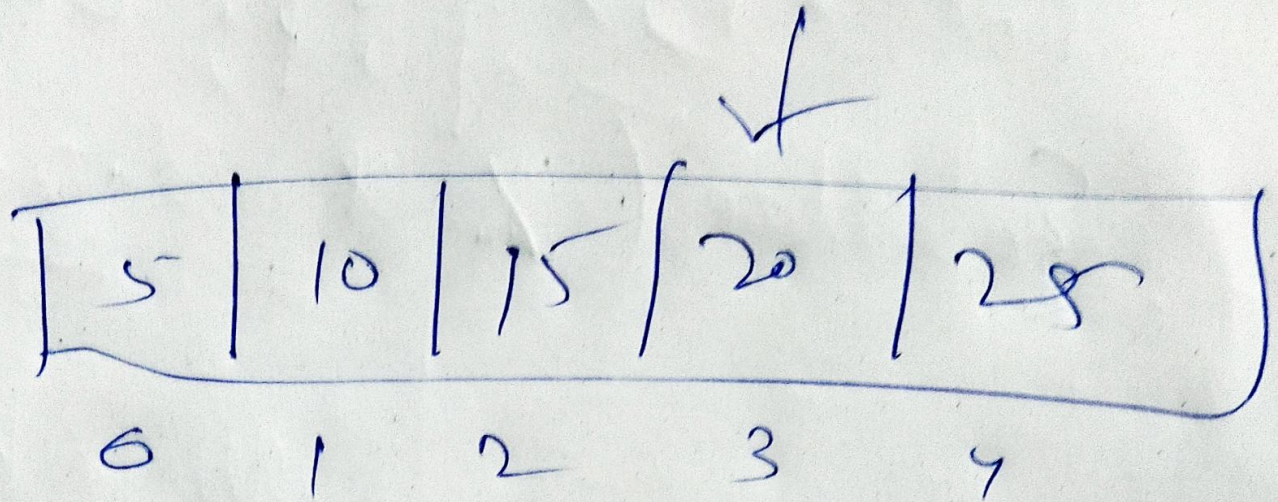
argc = 3

element
present

Pointer of Array

`int *a[5]`

{5, 10, 15, 20, 25};



`printf("%d", *a[3]);`

→ 20